

Machine Learning & Deep Learning

22-12-2025

Lab - 5

SVR

Importing the libraries

```
In [ ]: import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt
```

```
In [ ]: from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import StandardScaler  
from sklearn.svm import SVR
```

Read World bank CSV

```
In [ ]: df = pd.read_csv('WorldBank.csv')
```

```
In [ ]: df
```

**Perform conditional selection to find -
Population ages 15-64 (% of total
population)**

```
In [ ]: data = df[df['Indicator Name'] == 'Population ages 15-64 (% of total population)']
data
```

Divide the data into input and output

```
In [ ]: x = np.arange(1960,2020).reshape(-1,1)
y = data.values[0][4:-1].reshape(-1,1)
```

```
In [ ]: x
```

```
In [ ]: y
```

Plot scatter plot of Population ages 15-64 (% of total population)

```
In [ ]: plt.scatter(x,y)
plt.show()
```

Feature Scaling (Mandatory for SVR)**

SVR is highly sensitive to the range of data points. If we don't scale (normalize) the data, the model will fail to find the correct hyperplane.

```
In [ ]: sc_x = StandardScaler()
sc_y = StandardScaler()
```

```
In [ ]: x_scaled = sc_x.fit_transform(x)
y_scaled = sc_y.fit_transform(y)
```

```
In [ ]: x_scaled
```

```
In [ ]: y_scaled
```

```
In [ ]: print("Scaling Complete.")
```

Splitting the dataset into the Training set and Test set

```
In [ ]: x_train, x_test, y_train, y_test = train_test_split(x_scaled,y_scaled,random_state=
```

Fitting SVR on 3 Different Kernel on dataset

```
In [ ]: # revel function for the convert data into 1D from 2D
```

```
In [ ]: model_liner = SVR(kernel='linear')
model_poly = SVR(kernel='poly',degree=3)
model_rbf = SVR(kernel='rbf')
```

```
In [ ]: model_liner.fit(x_train,y_train.ravel())
model_poly.fit(x_train,y_train.ravel())
model_rbf.fit(x_train,y_train.ravel())
```

```
In [ ]: x_grid = np.arange(min(x_test),max(x_test),1)
x_grid = x_grid.reshape((len(x_grid),1))
# x_grid
```

```
In [ ]: plt.scatter(x,y,color = 'red',label = 'original data')

plt.plot(sc_x.inverse_transform(x_grid),
         sc_y.inverse_transform(model_rbf.predict(x_grid).reshape(-1,1)),
         color = 'blue',label = 'svr rbf')

plt.title('SVR Regression Result')
plt.xlabel('Year / Feature')
plt.ylabel('Population / target')
plt.legend()
plt.show()
```

Predict the x_test using 3 Kernel

```
In [ ]: print("Train")
print("Kernel Liner", model_liner.score(x_train,y_train))
print("Kernel Poly",model_poly.score(x_train,y_train))
print("Kernel RBF",model_rbf.score(x_train,y_train))
```

```
In [ ]: print("Test")
print("Kernel Liner", model_liner.score(x_test,y_test))
print("Kernel Poly",model_poly.score(x_test,y_test))
print("Kernel RBF",model_rbf.score(x_test,y_test))
```

Visualising the results

```
In [ ]: plt.figure(figsize=(12,8))
plt.scatter(x,y,color = 'orange',label = 'original data')
plt.plot(sc_x.inverse_transform(x_grid),
         sc_y.inverse_transform(model_rbf.predict(x_grid).reshape(-1,1)),
```

```

        color = 'blue',label = 'svr rbf')

plt.plot(sc_x.inverse_transform(x_grid),
         sc_y.inverse_transform(model_liner.predict(x_grid).reshape(-1,1)),
         color = 'red',label = 'svr liner')

plt.plot(sc_x.inverse_transform(x_grid),
         sc_y.inverse_transform(model_poly.predict(x_grid).reshape(-1,1)),
         color = 'green',label = 'svr poly')

plt.title('SVR Regression Result')
plt.xlabel('Year / Feature')
plt.ylabel('Population / target')
plt.legend()
plt.show()

```

Student Activity : Prediction

Task: Predict the value for the Year **2025** (or value 6.5 in the demo data). Remember: You must transform the input before predicting, and inverse transform the output.

```

In [ ]: value_to_predict = [[2025]]

val_scaled = sc_x.transform(value_to_predict)

predict_scaled_liner = model_liner.predict(val_scaled)
predict_scaled_poly = model_liner.predict(val_scaled)
predict_scaled_rbf = model_liner.predict(val_scaled)

predict_final_liner = sc_y.inverse_transform(predict_scaled_liner.reshape(-1,1))
predict_final_poly = sc_y.inverse_transform(predict_scaled_poly.reshape(-1,1))
predict_final_rbf = sc_y.inverse_transform(predict_scaled_rbf.reshape(-1,1))

print(predict_final_liner)
print(predict_final_poly)
print(predict_final_rbf)

```